Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

За шестой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:
Студент 3 курса
Группы ИИ-26(1)
Рулько М.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     А. С фиксированным шагом обучения
     Б. С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).

Perceptron.ts:

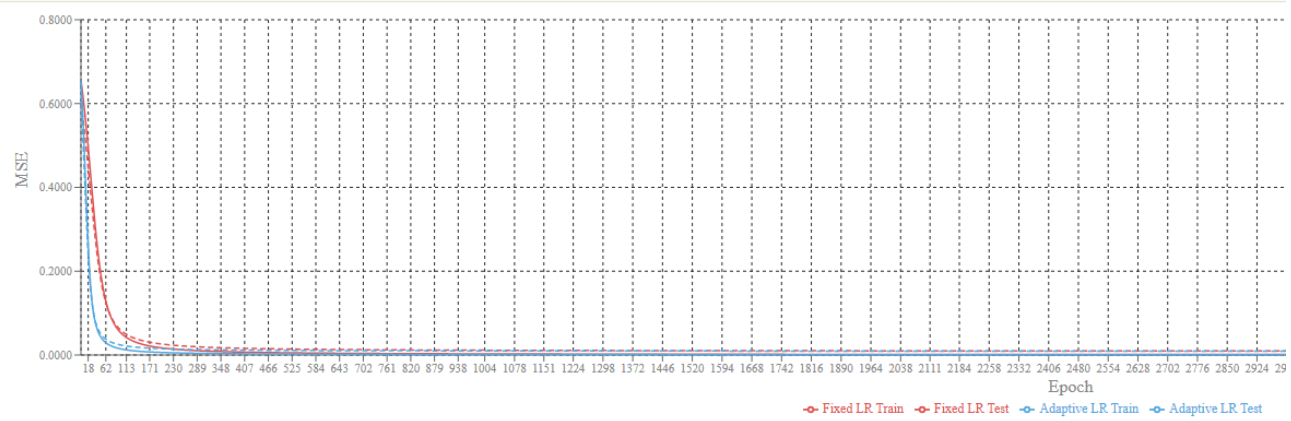
```
export type TrainResult =
{

  trainHistory: number[];
  testHistory: number[];
  weights: number[];
  threshold: number;
  epochs: number;

};

 function dot(a: number[], b: number[]): number
 {
  return a.reduce((s, v, i) => s + v * b[i], 0);
 }
//функция активации
 function sigmoid(x: number): number
 {
```

```
    return 1 / (1 + Math.exp(-x));
  }

  function insertBias(X: number[][]): number[][]
  {
    return X.map(row => [-1, ...row]);
  }
//среднеквадратичная ошибка
  function mse(y: number[], target: number[]): number
  {
    return y.reduce((s, v, i) => s + (v - target[i]) ** 2, 0) / y.length;
  }

  function predict(X: number[][], w: number[]): number[]
  {
    return X.map(row => sigmoid(dot(row, w)));
  }

function delta(
  X: number[][],
  y: number[],
  target: number[],
  w: number[],
  lr: number
): number[] {
  const grad = new Array(w.length).fill(0);
  for (let i = 0; i < y.length; i++) {
    const err = y[i] - target[i];
//производная сигмоиды
    const der = y[i] * (1 - y[i]);
    for (let j = 0; j < w.length; j++) {
      grad[j] += err * der * X[i][j];
    }
  }
  return w.map((wj, j) => wj - lr * (grad[j] / y.length));
}

function adaptiveLR(X: number[][]): number {
  const sumSq = X.reduce((s, row) => s + row.reduce((rs, v) => rs + v * v, 0), 0);
  return 1 / (sumSq / X.length);
}

export function generateData(N: number): { X: number[][]; Y: number[] } {
  const total = 1 << N;
  const X: number[][] = [];
  const Y: number[] = [];
  const inverseMask = [1, 0, 0, 0, 0, 0, 0, 0, 0].slice(0, N);

  for (let i = 0; i < total; i++) {
    const row: number[] = [];
    for (let b = N - 1; b >= 0; b--) {
```

```
      row.push((i >> b) & 1);
    }
    X.push(row);

    const modified = row.map((v, j) => (inverseMask[j] === 1 ? 1 - v : v));
    const target = modified.reduce((acc, v) => acc & v, 1);
    Y.push(target);
  }

  return { X, Y };
}

export function splitData(
  X: number[][],
  Y: number[],
  trainRatio = 0.9
): { XTrain: number[][]; YTrain: number[]; XTest: number[][]; YTest: number[] } {
  const onesIdx = Y.map((v, i) => (v === 1 ? i : -1)).filter(i => i >= 0);
  const zerosIdx = Y.map((v, i) => (v === 0 ? i : -1)).filter(i => i >= 0);

  const zerosShuffled = [...zerosIdx].sort(() => Math.random() - 0.5);
  const split = Math.floor(trainRatio * zerosShuffled.length);
  const trainIdx = [...onesIdx, ...zerosShuffled.slice(0, split)];
  const testIdx = zerosShuffled.slice(split);

  return {
    XTrain: trainIdx.map(i => X[i]),
    YTrain: trainIdx.map(i => Y[i]),
    XTest: testIdx.map(i => X[i]),
    YTest: testIdx.map(i => Y[i]),
  };
}

export function trainFixed(
  XTrain: number[][],
  YTrain: number[],
  XTest: number[][],
  YTest: number[],
  initialWeights: number[],
  lr: number,
  epochs: number,
  patience: number,
  targetAccuracy: number
):
TrainResult
{
  const XBias = insertBias(XTrain);
  const XTestBias = insertBias(XTest);
  let w = [...initialWeights];
  const trainHistory: number[] = [];
  const testHistory: number[] = [];
```

```
  let lastEpoch = epochs;

  for (let epoch = 0; epoch < epochs; epoch++) {
    const y = predict(XBias, w);
    const err = mse(y, YTrain);
    trainHistory.push(err);
    testHistory.push(mse(predict(XTestBias, w), YTest));

    if (err <= targetAccuracy) {
      lastEpoch = epoch + 1;
      break;
    }
//нет прогресса на последних patience эпохах
    if (trainHistory.length > patience) {
      const recent = trainHistory.slice(-patience);
      if (Math.max(...recent) - Math.min(...recent) < targetAccuracy) {
        lastEpoch = epoch + 1;
        break;
      }
    }

    w = delta(XBias, y, YTrain, w, lr);
  }

  return {
    trainHistory,
    testHistory,
    weights: w.slice(1),
    threshold: w[0],
    epochs: lastEpoch,
  };
}

export function trainAdaptive(
  XTrain: number[][],
  YTrain: number[],
  XTest: number[][],
  YTest: number[],
  initialWeights: number[],
  epochs: number,
  patience: number,
  targetAccuracy: number
):
TrainResult
{
  const XBias = insertBias(XTrain);
  const XTestBias = insertBias(XTest);
  let w = [...initialWeights];
  const lr = adaptiveLR(XBias);
  const trainHistory: number[] = [];
  const testHistory: number[] = [];
```

```
  let lastEpoch = epochs;

  for (let epoch = 0; epoch < epochs; epoch++) {
    const y = predict(XBias, w);
    const err = mse(y, YTrain);
    trainHistory.push(err);
    testHistory.push(mse(predict(XTestBias, w), YTest));

    if (err <= targetAccuracy) {
      lastEpoch = epoch + 1;
      break;
    }

    if (trainHistory.length > patience) {
      const recent = trainHistory.slice(-patience);
      if (Math.max(...recent) - Math.min(...recent) < targetAccuracy) {
        lastEpoch = epoch + 1;
        break;
      }
    }

    w = delta(XBias, y, YTrain, w, lr);
  }

  return {
    trainHistory,
    testHistory,
    weights: w.slice(1),
    threshold: w[0],
    epochs: lastEpoch,
  };
}

export function runInference(input: number[], weights: number[], threshold:
number): { prob: number; cls: number }
{
  const w = [threshold, ...weights];
  const x = [-1, ...input];
  const prob = sigmoid(dot(x, w));
//выбираем какой класс
  return { prob, cls: prob > 0.5 ? 1 : 0 };
}
```




```
> perceptron.run([0,1,1,1,1,1,1,1,1])
Input:       [0, 1, 1, 1, 1, 1, 1, 1, 1]                App.tsx:34
Probability: 0.004116                                   App.tsx:35
Class:       0                                          App.tsx:36
<· ▸ {prob: 0.004116099760477867, cls: 0}
```

Вывод: реализовал нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.